

CABSITE Backend

Setup, deployment and operations. Fanart gallery with Discord login and a moderator queue, running as a single Cloudflare Worker.

1. What was built

The existing static site and a new backend now ship as **one Cloudflare Worker**. There is no second server, no separate origin, and no CORS. The DigitalOcean droplet is not involved.

LAYER	TECHNOLOGY	ROLE
Static pages	Workers Assets	Serves everything in <code>public/</code>
Backend	TypeScript + Hono	<code>/auth/</code> , <code>/api/</code> , <code>/art/</code>
Sessions	KV	Session id → user id, 7 day TTL
Relational data	D1	Users, posts, roles
Image files	R2	Uploaded art, free egress
Login	Discord OAuth	No passwords stored
Bot protection	Turnstile	On the upload route

The rule that drives the design

Nothing becomes public without a moderator approving it. Uploads land as `pending`. Pending images return 404 to everyone except moderators and the owner. Rejection deletes the database row and the R2 object — there is no third status and no archive.

2. Where everything lives

```
cabini/
├─ public/           ← served to browsers, this is the web root
│  ├─ index.html     the original shrine, unchanged
│  ├─ style.css  script.js  favicon.ico  media/
│  ├─ robots.txt  sitemap.xml
│  ├─ gallery.html  + gallery.js  approved art
│  ├─ upload.html  + upload.js  submission form
│  ├─ admin.html   + admin.js  moderator queue
│  ├─ owner.html   + owner.js  staff management (owner only)
│  └─ nav-auth.js  shared nav + sign in state
├─ src/             ← the Worker, never served to browsers
│  ├─ index.ts      routing, /api/me, /art/* image serving
│  ├─ auth.ts       Discord OAuth
│  ├─ session.ts    cookie → KV → D1, role guards
│  ├─ cookie.ts     HMAC signing
│  ├─ posts.ts      gallery list + upload
│  ├─ admin.ts      moderator queue
│  ├─ owner.ts      promote / demote moderators
│  ├─ turnstile.ts  image.ts  types.ts
├─ schema.sql       database tables
├─ wrangler.jsonc   config + public vars
├─ .dev.vars        local secrets, gitignored
└─ .wrangler/       local database + files, gitignored
```

Anything you put in `public/` is publicly downloadable. That is the whole point of the folder, but it means secrets, notes and backups must never go there. Worker code in `src/` is compiled and never exposed.

3. Permission tiers

TIER	HOW IT IS GRANTED	CAN DO
Anyone	No account	View the shrine and approved art
Member	Automatic on first Discord login	Submit art for review
Moderator	Granted by the owner on the Staff page	Approve or reject submissions, preview pending art
Owner	Discord ID in <code>wrangler.jsonc</code>	Everything a moderator can, plus add and remove moderators

The owner is **you**, pinned to Discord ID `733654151107444797`. This is deliberately *not* stored as a database role:

- **It cannot be taken.** Anyone who gained write access to the database still could not make themselves owner — the check reads config, not a table.
- **It cannot lock you out.** On an empty database there are no moderators yet. Because owner comes from config, you can sign in and appoint the first one immediately.
- **It survives a database wipe.** Restore from nothing and you are still owner.

The consequence: your row in the users table says `member`, and that is correct. Owner status is layered on top at request time.

4. Setup you still have to do

Everything below is required before the site works in production. Local development already works with none of it.

STEP 1 Create the Discord application

1. Go to `discord.com/developers/applications` and click **New Application**.
2. Open **OAuth2**. Copy the **Client ID** and reset/copy the **Client Secret**.
3. Under **Redirects**, add *both* of these, exactly:

```
http://localhost:8787/auth/callback
https://cabinithepanini.com/auth/callback
```

These must match character for character — including `http` vs `https` and no trailing slash. A mismatch gives an `invalid_redirect_uri` error at login and is the single most common thing to get wrong here.

STEP 2 Create a Turnstile widget

1. Cloudflare dashboard → **Turnstile** → **Add site**.
2. Hostname `cabinithepanini.com` . Add `localhost` too if you want the real widget locally.
3. Copy the **Site Key** (public) and **Secret Key** (private).

STEP 3 Create the Cloudflare resources

```
npx wrangler d1 create cabsite
npx wrangler kv namespace create SESSIONS
npx wrangler r2 bucket create cabsite-art
```

Each command prints an ID. Paste them into `wrangler.jsonc` , replacing the `"local"` placeholders.

R2 is the one product that asks for billing details up front. You will not be charged — 10 GB of storage is free and egress is always free — but the card has to be on file before the bucket can be created. Everything else is on the free plan. This is why local development was built to need none of it.

STEP 4 Fill in the public config

In `wrangler.jsonc` , under `vars` . These are *not* secrets and are safe to commit:

```
"vars": {
  "DISCORD_CLIENT_ID": "your client id from step 1",
  "DISCORD_REDIRECT_URI": "https://cabinithepanini.com/auth/callback",
  "TURNSTILE_SITE_KEY": "your site key from step 2",
  "OWNER_DISCORD_ID": "733654151107444797"
}
```

Two values ship as local-development defaults and will break production if left alone.

`DISCORD_REDIRECT_URI` is currently `http://localhost:8787/auth/callback` . Deploy that and every login fails.

`TURNSTILE_SITE_KEY` is currently `1x00000000000000000000AA` , Cloudflare's test key that always passes. Deploy that and the bot check is decorative.

STEP 5 Set the secrets

These are encrypted by Cloudflare and never appear in the repository:

```
npx wrangler secret put DISCORD_CLIENT_SECRET
npx wrangler secret put TURNSTILE_SECRET
npx wrangler secret put COOKIE_SECRET
```

For `COOKIE_SECRET`, paste a long random string. Generate one with:

```
node -e "console.log(crypto.randomUUID() + crypto.randomUUID())"
```

This key signs session cookies. Changing it later signs everyone out — which is also how you force a global logout if a session is ever compromised.

STEP 6 Create the tables and deploy

```
npx wrangler d1 execute cabsite --remote --file=./schema.sql
npx wrangler deploy
```

`schema.sql` begins with `DROP TABLE IF EXISTS`. Running it against a live database **erases every user and post**. Run it once at setup. After that, write migrations instead.

STEP 7 Appoint your moderators

1. Visit <https://cabinthebanini.com/gallery> and sign in with Discord.
2. A **Staff** link appears in the nav. It is visible only to you.
3. Have your moderators sign in once so they have an account.
4. On the Staff page, click **Make moderator** next to each.

Role changes apply on their next request. Nobody needs to sign out and back in.

5. Routes

ROUTE	WHO	WHAT IT DOES
GET /auth/discord	Anyone	Start Discord login
GET /auth/callback	Anyone	Finish login, create session
POST /auth/logout	Anyone	Destroy session
GET /api/me	Anyone	Current user, role, owner flag
GET /api/posts	Anyone	Approved art
POST /api/posts	Members	Turnstile check, then store as pending
GET /art/*	Anyone	Approved images; pending visible to staff only
GET /api/admin/posts	Moderators	The pending queue
POST /api/admin/posts/:id/approve	Moderators	Publish, record who and when
DELETE /api/admin/posts/:id	Moderators	Delete row and image
GET /api/owner/users	Owner	All accounts and roles
POST /api/owner/users/:id/role	Owner	Promote or demote

6. Upload rules

LIMIT	VALUE	RESPONSE IF BROKEN
File size	5 MB	413
Formats	png, jpeg, gif, webp	415
Title	1–120 characters	400
Description	up to 1000 characters	400
Bot check	Turnstile token required	403

File type is decided by reading the first bytes of the file, not by trusting what the browser claims. Renaming `virus.exe` to `art.png` does not get past it. To change the size cap, edit `MAX_BYTES` in `src/image.ts`.

7. Day to day operations

Local development

```
npm install
npm run db:init      # create tables in the local database
npm run dev          # http://localhost:8787
```

Local mode simulates D1, KV and R2 on disk under `.wrangler/state`. Nothing touches your Cloudflare account and nothing is billed. To wipe local data and start clean:

```
rm -rf .wrangler/state
npm run db:init
```

Promote someone without the web page

```
npx wrangler d1 execute cabsite --remote \
  --command "UPDATE users SET role='moderator' WHERE username='NAME';"
```

Look at the live data

```
npx wrangler d1 execute cabsite --remote --command "SELECT username, role FROM users;"
npx wrangler d1 execute cabsite --remote --command "SELECT COUNT(*) FROM posts WHERE
status='pending';"
npx wrangler tail                                # live request logs
```

Back up the database

```
npx wrangler d1 export cabsite --remote --output=backup.sql
```

D1 keeps 30 days of point-in-time recovery on paid plans. Manual exports are worth taking before any schema change.

8. Deliberately not built

ITEM	STATUS
AI moderation	Deferred. Slots in between the Turnstile check and the database insert. If added, it may only auto-approve or defer to the queue — never auto-reject.
Comments, likes, follows	Out of scope
Email	Never. Discord identity only.
Auto-publishing	Never. A human approves every image.
Editing a submission	Not built. Reject it and have them resubmit.
Pagination	Not built. Both lists cap at 100 rows, which is fine until the gallery grows past that.
Deleting a user account	Not built. Only role changes are exposed.

9. When something breaks

SYMPTOM	CAUSE AND FIX
<code>invalid_redirect_uri</code> at login	The Discord redirect list does not exactly match <code>DISCORD_REDIRECT_URI</code> . Compare both, character for character.
Signed in, but treated as signed out	<code>COOKIE_SECRET</code> changed, or the 7 day session expired. Sign in again.
No Staff link in the nav	<code>OWNER_DISCORD_ID</code> does not match the account you logged in with. Check <code>/api/me</code> and compare <code>discord_id</code> .
Every upload fails the bot check	<code>TURNSTILE_SECRET</code> and <code>TURNSTILE_SITE_KEY</code> are from different widgets, or the site key is still the test value.
Gallery loads but images are blank	Those posts are still pending, or the R2 bucket ID in <code>wrangler.jsonc</code> is wrong.
Anything unexplained	<code>npx wrangler tail</code> streams live errors from production.

10. Costs

Free to run at your scale. Workers allows 100,000 requests a day on the free plan; the paid plan is \$5 a month if you outgrow it. R2 gives 10 GB free, then \$0.015 per GB per month, and egress is always free — which is what matters for an image gallery. D1, KV and Turnstile are all covered by the free plan.

At 5 MB per image, 10 GB holds roughly 2,000 submissions before storage costs anything at all.

